

Degenerate-Set Annihilation: Predicting Fixed-Point Error in FPGA ADMM Solvers

Anonymous Authors
Submission for double-blind review

Abstract—Fixed-point wordlength selection for FPGA optimization solvers rests on the classical model in which every quantization contributes independent noise of power $q^2/12$. We show that this model is wrong for proximal operators in a specific, measurable way: on the operator’s *degenerate set*, where its derivative vanishes, injected quantization error is annihilated exactly, so effective error scales with $(1 - d)$ for degenerate fraction d . The resulting model matches bit-exact measurement to 0.99 (soft-threshold) and 1.00 (box projection) on average over $d \in [0.12, 0.88]$, where the classical error grows as $1/(1 - d)$: $3\text{--}4\times$ averaged over that range, $900\times$ at $d = 0.999$. It transfers unchanged to a second problem, LASSO, at 0.98–1.03 with d up to 0.77. It explains the datapath width that a bit-exact sweep selects for an ADMM accelerator on Artix-7, which we measure on routed silicon: narrowing Q2.16 to Q2.9 removes 44.9% of LUTs, raises $F_{\max}$ by 28.8%, and cuts dynamic energy per solve from 184 to 117 nJ, from activity-annotated power, with detection BER statistically unchanged. Both designs run bit-exact on a board at their in-context $F_{\max}$, where the speed-up is 26.9%. Narrowing a single operator’s lane, by contrast, costs 3.7–4.1% area at two widths and buys no resolvable power or frequency. We state where the model stops: it predicts ensemble behaviour, not per-instance worst cases.

I. INTRODUCTION

An FPGA accelerator for a proximal splitting method such as ADMM [1] must fix a number format for every signal. Too few fraction bits and the solver returns a wrong answer; too many and area, critical path and switching energy all grow. Designers therefore either simulate exhaustively or reason with the classical model, in which each rounding is an independent additive source of power $q^2/12$ [2] propagated through the algorithm.

That model is known to be inaccurate in feedback loops and under correlated quantizations [3], [4], and the incumbents of recent search-based wordlength optimization note that analytical noise models remain limited to linear time-invariant systems [5], [6]. A proximal operator is neither linear nor smooth. Jerez et al., analysing fixed-point ADMM on FPGA, observe that the projection step can only *reduce* error, by a diagonal factor with entries in $[0, 1]$ — and then set that factor to the identity to keep their bound conservative [7]. Kinsman and Nicolici note that their bit-width constraints assume no correlation between error and value, and that a known correlation “can be captured into constraints” [8].

This paper quantifies the factor that was discarded and supplies the correlation that was missing. Our contributions:

- 1) **An operator error model** in which the degenerate set of a proximal operator annihilates a measurable fraction

$(1 - d)$ of injected quantization error (Theorem 1), validated to 0.99/1.00 against bit-exact simulation, with a stated 12% miss on the coefficient term of the one operator without a degenerate set.

- 2) **Transfer to a second problem.** The same model holds on LASSO’s native distribution (0.98–1.03, d up to 0.77, off-lattice threshold), and the accelerator runs LASSO bit-exact, on a board, with no change.
- 3) **A measured design point.** Selected by bit-exact sweep and explained by the model, a Q2.9 datapath on Artix-7 saves 44.9% of LUTs and 36% of dynamic energy per solve against Q2.16, with unchanged detection BER, at 28.8% higher $F_{\max}$ out of context and 26.9% higher in context, bit-exact on a board at speed.
- 4) **A negative result.** Narrowing one operator’s lane below the main width — the asymmetry an operator-level model invites — costs area at both widths tested and buys no resolvable power or frequency.

We do not claim the first error model for fixed-point ADMM (Jerez et al. have one), a worst-case bound, or that the width follows analytically without simulation. Section IX states where the model stops.

II. BACKGROUND AND RELATED WORK

a) **ADMM with proximal lanes:** For $\min_x \frac{1}{2} \|Hx - y\|^2 + g(x)$ in scaled form,

$$\begin{aligned} x^{k+1} &= M(q + \rho(z^k - u^k)), \\ z^{k+1} &= \text{prox}_{g/\rho}(x^{k+1} + u^k), \\ u^{k+1} &= u^k + x^{k+1} - z^{k+1}. \end{aligned} \quad (1)$$

with $M = (H^\top H + \rho I)^{-1}$ and $q = H^\top y$. The x-update is a dense mat-vec with a matrix fixed per problem; the z-update is elementwise. We study three proximal operators [9]: soft-threshold $S_\kappa(v) = \text{sgn}(v) \max(|v| - \kappa, 0)$ (L1), projection onto $[\ell, h]$ (Box), and ridge scaling γv (L2).

b) **Error analysis of fixed-point ADMM:** Jerez et al. derive a converging upper bound on accumulated round-off for fixed-point ADMM on FPGA and use it to choose a uniform fraction width [7]; their bound discards the projection’s attenuating factor, which is the quantity we measure. The Heriot-Watt line bounds inexact proximal methods under errors assumed additive, bounded and, in [10], zero-mean and orthogonal to the iterate [10], [11]. Degenerate-set annihilation violates that orthogonality by construction: which coordinates are annihilated depends on the iterate. Our model is a refinement of theirs, and it is predictive where theirs are bounds.

c) *Wordlength optimization*: Analytical methods for LTI systems [12] and search methods for general ones [5], [6] are mature; for iterative computations, Kinsman and Nicolici allocate widths by SMT with conjugate gradient as the case study, and their robust widths reach single to double precision after input-space restriction [8]. Li et al. set the methodological bar we adopt — an analytical model validated against bit-exact simulation to under 1 dB, with measured hardware — for a different, linear algorithm [13].

d) *Mixed precision*: Wu et al. report up to 67% LUT and 60% power reduction for mixed-precision ADMM [14]. Those figures are against an FP32 baseline, with power from the vendor estimator, and vary precision over iterations; ours vary it over operators, against a fixed-point baseline, with activity-annotated power. In their fixed-point ADMM rows the mixed configuration saves DSPs (21 vs. 30) but uses slightly more LUTs (6807 vs. 6775): non-uniform precision is not free in fabric there either.

III. THE ERROR MODEL

A. Degenerate-set annihilation

Let P_θ be a proximal operator with parameter θ , applied elementwise and evaluated in a lane with W fraction bits, $q = 2^{-W}$. Split the input coordinates into the degenerate set $D = \{i : \partial P / \partial v_i = 0\}$ and its complement, and write $d = |D|/n$ and L for the slope on D^c .

Theorem 1 (Operator error decomposition). *Let the lane round its input to the lattice, $\tilde{v} = v + e$ with e uniform on $[-q/2, q/2)$ and independent of v ; let P be piecewise affine in v with finitely many breakpoints, slope 0 on D and L on D^c ; let v have a bounded density near the breakpoints, with $d = \Pr(v \in D)$; and let θ be held as $\tilde{\theta} = \theta + \delta_\theta$ with $\delta_\theta = O(q)$. Then the per-coordinate output error $e_P = P_{\tilde{\theta}}(\tilde{v}) - P_\theta(v)$ satisfies*

$$\mathbb{E} e_P^2 = (1-d) \left[L^2 \frac{q^2}{12} + \sigma_{\theta, D^c}^2 \right] + d \sigma_{\theta, D}^2 + O(q^3), \quad (2)$$

where $\sigma_{\theta, S}^2 = \mathbb{E}[(\partial P / \partial \theta)^2 | v \in S] \delta_\theta^2$. If the output is not on the lattice (a product, as in L2), add its rounding, $q^2/12$.

Proof. Condition on v . If v lies in the interior of D^c at distance more than $q/2 + |\delta_\theta|$ from every breakpoint (so that neither the input nor the parameter rounding moves v across one), $\tilde{v}$ stays on the same affine piece and $e_P = L e + (\partial P / \partial \theta) \delta_\theta$; since $\mathbb{E} e = 0$ the cross term vanishes and $\mathbb{E}[e_P^2 | v] = L^2 q^2/12 + (\partial P / \partial \theta)^2 \delta_\theta^2$. If v lies in the interior of D , equally far from every breakpoint, P is constant in v on the whole interval reachable by e , so the input error is removed exactly and only $(\partial P / \partial \theta) \delta_\theta$ remains. The remaining inputs lie within $q/2$ of a breakpoint, a set of probability $O(q)$ by the density bound, on which $|e_P| \leq L q/2 + |\partial P / \partial \theta| |\delta_\theta|$; with $\delta_\theta = O(q)$ they contribute $O(q^3)$. Averaging over v gives (2). $\square$

The theorem is operator-level: one application of P to an input carrying fresh rounding error, which is how a lane sees it each iteration; error accumulated around the loop is the

TABLE I
THEOREM 1 VS. BIT-EXACT MEASUREMENT

operator	input distribution	d	T1/meas	classical/meas
L1	sweep, on-lattice	0.17–0.88	0.991	$3.8\times$
Box	sweep, on-lattice	0.12–0.83	1.004	$3.0\times$
Box	sweep, on-lattice	0.999	0.96	$903\times$
L1	detection solver	0.00	0.99	0.61^a
Box	detection solver	0.18	1.00	0.95
L2	detection solver	0	0.88	0.21^a
L1	LASSO solver, 5 λ	0.62–0.77	0.98–1.03	$1.6\text{--}2.7\times$

^aBelow 1: with $d \approx 0$ there is nothing to annihilate, and parameter rounding (κ , γ) adds error the classical model omits.

subject of Sect. III-D. Equivalently, $(1-d)L^2 = \mathbb{E}[P'(v)^2]$ is the operator's first-order perturbation gain. What matters for a proximal operator is that this expectation is set by d , which the data, the parameter and the iterate determine, and which in our workloads ranges from 0 to above 0.9. Concretely, with δ_θ the rounding error of θ on the lane lattice: for L1, $L = 1$, $\sigma_{\theta, D^c}^2 = \delta_\kappa^2$ and $\sigma_{\theta, D}^2 = 0$; for Box, $L = 1$, $\sigma_{\theta, D^c}^2 = 0$ and $\sigma_{\theta, D}^2$ is the squared rounding error of whichever bound clips; for L2, $d = 0$, $L = \gamma$, and the coefficient error enters as $\mathbb{E}[v^2] \delta_\gamma^2$. The classical model charges every coordinate $L^2 q^2/12$, so with an on-lattice parameter it overestimates by $1/(1-d)$. That is not a constant correction: d is set by the data, the parameter and the iterate.

B. Validation against bit-exact simulation

We measure each operator in isolation, on inputs harvested from floating-point ADMM runs, against a bit-exact model of the RTL (Fig. 1). Sweeping each operator's parameter moves d across $[0, 1]$; parameters are placed on the lattice so that the θ terms vanish and annihilation is isolated. Over $d \in [0.17, 0.88]$ for L1 and $[0.12, 0.83]$ for Box, prediction/measured is 0.991 and 1.004 on average (0.979–1.020 per point), while the classical model is off by $3.8\times$ and $3.0\times$ on average — $1.1\text{--}1.2\times$ at the low end of the range, $903\times$ at $d = 0.999$, and infinitely at $d = 1$, where the measured error is exactly zero. Very close to $d = 1$ the residual error is a handful of boundary events and the ratio becomes noisy (0.96 at $d = 0.999$, 0.69 at $d = 0.9999$). On the solver's own operating distribution, averaged over $W = 6\text{--}12$, the model gives 0.99 (L1) and 1.00 (Box). For L2 it gives 0.88: with $d = 0$ the annihilation term is inactive and the 12% miss lies in the coefficient term, which this result does not validate to better than 12%. In bits, the model errs by 0.007 (L1), 0.000 (Box) and 0.092 (L2), inside the 0.17-bit (1 dB) bar of [13]. Table I collects the validation.

C. Two consequences for design

a) *Lattice rule*: If θ is not on the lane's lattice, its rounding error δ enters every coordinate where the output depends on θ , and for $|\delta| \sim q/2$ it dominates (2). Placing operator constants on the lattice is worth one bit: measured $3.68\text{--}3.90\times$ error against a predicted $4\times$, and one bit through the closed loop.

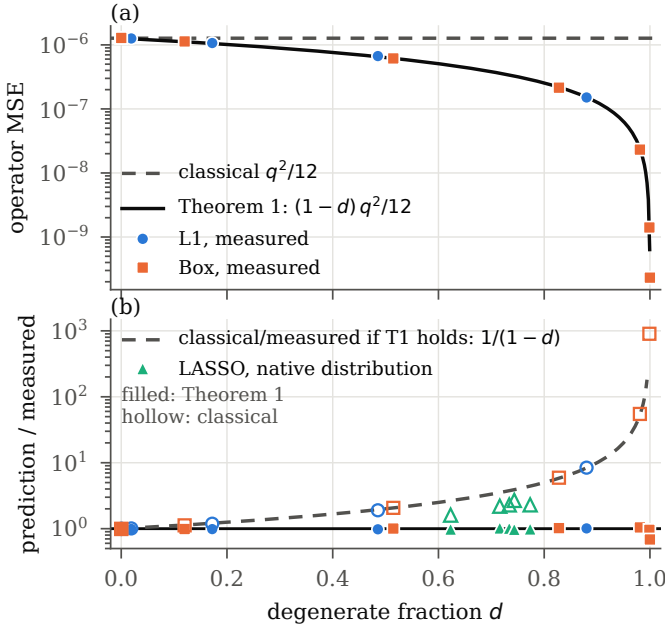


Fig. 1. Theorem 1 against bit-exact measurement. (a) L1 and Box error as each operator’s parameter moves d across $[0, 1]$ ($W = 8$, on-lattice): measurement follows $(1-d)q^2/12$, not the classical flat $q^2/12$; at $d = 1$ it is exactly zero (not plotted on the log axis). (b) The same as ratios, with LASSO on its native distribution (Sect. VI-F), where the off-lattice threshold adds a parameter term that the model includes and the classical model does not.

b) Range feasibility: The datapath must represent $M = (H^T H + \rho I)^{-1}$, whose entries satisfy $|M_{ij}| \leq 1/\rho$, a bound approached as $H^T H$ becomes singular.

Proposition 1. *Representing M in QI.F for every H requires $\rho > 2^{-(I-1)}$; with $I = 2$, $\rho > 0.5$ (or $\rho = 0.5$ with one-LSB saturation of M).*

We registered this before measuring it. On the detection workload, $\max |M|$ is 2.91 at $\rho = 0.125$ (unrepresentable in Q2.F) and 1.34 at $\rho = 0.5$; on an underdetermined generator, where $H^T H$ is singular, it reaches 1.99 at $\rho = 0.5$, against the bound of 2. We use $\rho = 1$.

D. From operator to solver

Operator error is amplified by the ADMM loop by a gain G . For the contractive L2 lane, $\log G = a \log R + c$ with $R = \|(I - T)^{-1}\|$ the resolvent of the linearized iteration predicts solver-level error to 0.046 bits on problem instances *disjoint* from those it was fitted on. The gain itself is the point, more than its variation: G moves only between 6.4 and 12.1 across $300\times$ of conditioning, because contraction pins the resolvent, so a single constant already does well (0.094 bits); ignoring the loop ($G = 1$) under-predicts by 1.65 bits. Adding one bit to the prediction covers 99% of random held-out instances (+0.66 at the 99th percentile), but not an adversarial search, which finds +1.12. For L1 and Box, fitted loop-gain forms that looked accurate under cross-validation on the fitting instances (0.34 and 0.23 bits) fail on disjoint instances (1.38 and 0.73 bits); we retract them and

TABLE II
LOOP-GAIN MODEL: FITTED VS. DISJOINT INSTANCES (BITS)

lane	in-sample	held-out	instance-set s.d.	p99 per instance
L1	0.279	1.382	0.593	+1.90
Box	0.146	0.734	0.156	+3.46
L2	0.061	0.046	0.075	+0.66

TABLE III
MINIMUM MAIN WIDTH F MEETING AN NMSE TARGET (NO SATURATION)

NMSE target	L1	Box	L2
10^{-3}	8	7	7
10^{-4}	9	9	9
10^{-5}	11	10	10
10^{-4} , lanes pinned 8/7/8	9	9	9

claim only the operator-level model for these lanes. Table II shows why cross-validation on the same instances could not see it. For L1, a single instance carries 68% of the ensemble error in the median cell, and the ensemble mean varies by 0.59 bits (s.d.) between instance sets — more than the fit’s own error. For Box, the held-out error is $4.7\times$ its instance-set variability: the fit is over-fitted, not noisy. A criterion cannot be tighter than the quantity’s repeatability; we therefore hold loop-level claims to 0.5 bits on held-out instances, the resolution of an integer width decision.

IV. FROM MODEL TO DATAPATH

The model’s job is to say where to look and why; a bit-exact sweep confirms the choice. We sweep the main fraction width F with all lanes tied to it, on 8-variable problems (32×8 real BPSK detection, $\rho = 1$), and record the smallest F whose solution NMSE against double precision meets a target, with saturation counters required to stay at zero (otherwise the sweep measures clipping, not quantization).

Table III gives the result. At NMSE 10^{-4} every operator needs $F = 9$; at 10^{-3} , $F = 8/7/7$ for L1/Box/L2; at 10^{-5} , $11/10/10$.

We take Q2.9 (11 bits) as the design point. The operators differ by at most one bit at any target. At 10^{-4} , lanes pinned at 8/7/8 need exactly the same F as lanes tied to it, so narrowing individual lanes costs no accuracy there; at 10^{-5} the pinned lanes cannot reach the target at any F , and lane width binds. Whether narrowing buys area is what Section VI-E tests in fabric. We make no claim here about which operator needs the most precision.

a) Application quality: NMSE against double precision is a proxy; detection accuracy is what the application sees. Table IV gives uncoded BER for the Box (detection) lane over 24 000 bits per SNR, through the same bit-exact datapath that was measured on silicon. We registered two predictions. Double-precision BER lies within the 95% confidence interval of the Q2.9 estimate at every SNR: it does. Q2.9 makes the same decision as double precision on at least 99.9% of bits: it

TABLE IV
UNCODED BER, BPSK 32×8 , 24 000 BITS PER SNR

SNR (dB)	ridge ^a	ADMM, double	Q2.16	Q2.9
0	$4.06 \cdot 10^{-2}$	$3.16 \cdot 10^{-2}$	$3.16 \cdot 10^{-2}$	$3.22 \cdot 10^{-2}$
2	$2.00 \cdot 10^{-2}$	$1.26 \cdot 10^{-2}$	$1.26 \cdot 10^{-2}$	$1.28 \cdot 10^{-2}$
4	$8.42 \cdot 10^{-3}$	$2.42 \cdot 10^{-3}$	$2.42 \cdot 10^{-3}$	$2.42 \cdot 10^{-3}$
6	$4.17 \cdot 10^{-3}$	$3.33 \cdot 10^{-4}$	$3.33 \cdot 10^{-4}$	$3.75 \cdot 10^{-4}$
8	$1.37 \cdot 10^{-3}$	$4.17 \cdot 10^{-5}$	$4.17 \cdot 10^{-5}$	$4.17 \cdot 10^{-5}$
10–14	$\geq 7.1 \cdot 10^{-4}$	0	0	0

^aSigns of the x-update alone, $(H^T H + I)^{-1} H^T y$; a linear detector with fixed regularization, whose high-SNR floor is residual inter-symbol interference, not noise.

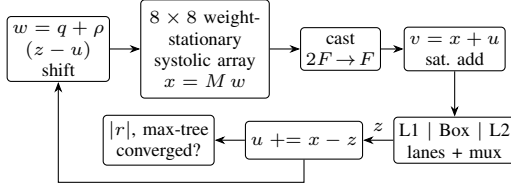


Fig. 2. Datapath. All signals Q2.F; products accumulate at $2F$ and are cast back. ρ is a power of two, so the pre-combination is a shift. Each iteration is 26 cycles; a solve of K iterations is $26K + 3$.

does at every SNR except 0 dB, where it agrees on 99.896% (25 bits of 24 000) — a miss as registered, at a BER of 3%.

In use, the model explains the sweep’s result and says where to look; it does not replace the sweep. Proposition 1 fixes ρ against the integer width; the lattice rule fixes the operator constants; Theorem 1, with d measured from a floating-point run, gives each lane’s operator-level error at a candidate W , and so which lane can be narrowed at what cost; for the contractive lane the loop model plus its one-bit margin gives the solver-level width. A bit-exact sweep over representative instances then confirms the choice, as Table III does here.

V. ARCHITECTURE

The accelerator (Fig. 2) maps (1) directly. M is loaded once per problem into an 8×8 weight-stationary systolic array whose input vector is pipelined down the columns and whose partial sums accumulate along the rows; ADMM is sequentially dependent, so there is one vector in flight and no input skew. With ρ a power of two the pre-combination is a shift. The three proximal lanes are all instantiated and selected at run time, so that a lane cannot be removed by constant propagation and inflate a reported saving. Convergence is tested on $\|r\|_\infty$ through a balanced max-tree; with the threshold scaled by $1/\sqrt{N}$ this is sufficient for the ℓ_2 criterion, and avoids N multipliers and a square root. Every arithmetic primitive saturates rather than wraps, and a bit-exact Python model mirrors each one.

Each iteration takes 26 cycles, the array’s $2N + 2 = 18$ plus the vector stages, and a solve of K iterations takes exactly $26K + 3$ cycles in every build we measured. Cycles per iteration are independent of every width parameter, so all of the speed difference between builds comes from $F_{\max}$. Our validation runs set the convergence threshold to zero, so the early exit fires when the residual quantizes to exactly zero —

TABLE V
Q2.16 VS. THE Q2.9 DESIGN POINT (ARTIX-7, ROUTED)

	Q2.16	Q2.9	change
Slice LUTs	4673	2574	−44.9%
proximal lanes	2159	1032	−52.2%
Flip-flops	2199	1359	−38.2%
DSP48 / BRAM	64 / 0	64 / 0	—
$F_{\max}$, out of context	66.3 MHz	85.4 MHz	+28.8%
$F_{\max}$, in context (board)	65 MHz	82.5 MHz	+26.9%
Dynamic power (SAIF)	11 mW	7 mW	—
Dynamic energy / solve, $K = 32$	184 nJ	117 nJ	−36%

stagnation, which comes sooner at coarser precision (15/19/12 iterations at Q2.9 against 28/32/22 at Q2.16). Those counts describe termination, not convergence, and we quote every throughput and energy figure at a fixed $K = 32$.

VI. HARDWARE RESULTS

A. Method

Artix-7 XC7A35T, Vivado 2025.2, out-of-context implementation of the core. Area is Slice LUTs from `report_utilization`, the metric comparators report. $F_{\max}$ is found by binary search on the clock constraint with a full implementation at each step, reported as $1/T$ for the tightest period T that met timing — never as $1/(T - \text{slack})$. Power is post-route, annotated with switching activity from a gate-level simulation of the regression vectors (SAIF), at Vivado’s *High* confidence with 72–75% of nets matched; every such simulation also checks the output bit-exact against the golden model. The vectorless estimate for the same builds was about $10\times$ too high. Reports round to 1 mW, which bounds every power ratio we quote. Every figure in this section is regenerated by committed scripts, and a fresh run reproduces all seven builds exactly.

B. The design point

Table V gives the result. Narrowing the datapath from Q2.16 to Q2.9 removes 44.9% of LUTs, more than half of them in the proximal lanes. $F_{\max}$ rises 28.8%; the binary search resolution bounds the ratio at +27.6% to +30.4%. Dynamic power falls from 11 to 7 mW; with 1 mW rounding the reduction lies between 29 and 44%. Dynamic energy per solve combines the measured power, the measured cycle count and the measurement clock (50 MHz): $P_{\text{dyn}} \cdot (26K + 3) / f$ gives 184 and 117 nJ at $K = 32$. It inherits the same rounding — the −36% is bounded at 29–44% — but not a dependence on clock: energy per cycle is frequency-independent, so the figure holds at whatever clock a reader assumes. Device static power is 68 mW in both builds, so *total* on-chip power falls only from 80 to 76 mW; we claim the dynamic reduction only.

The speed-up has a structural cause. Across all seven builds the critical path falls into two families with no overlap: through a proximal lane (21–24 logic levels, 63.9–68.9 MHz) or through the convergence test (16–17 levels, 79.7–86.5 MHz). At Q2.16 the lane binds; at Q2.9 the lanes are short enough that the convergence test binds instead. Narrowing a lane

TABLE VI

ALL SEVEN BUILDS (ROUTED; POWER FROM SAIF, 50 MHz). PATH: CRITICAL PATH THROUGH A PROXIMAL LANE (A) OR THE CONVERGENCE TEST (B).

F	lanes	LUT	lanes' LUT	FF	$F_{\max}$	dyn mW / path
16	16/16/16	4673	2159	2199	66.3	11 / A
16	8/8/8	3259	1020	1975	67.6	10 / A
16	8/7/8	3392	1153	1975	63.9	10 / A
10	10/10/10	3043	1272	1479	79.7	7 / B
10	8/8/8	2698	976	1447	68.9	7 / A
9	9/9/9	2574	1032	1359	85.4	7 / B
9	9/8/9	2669	1128	1359	86.5	7 / B

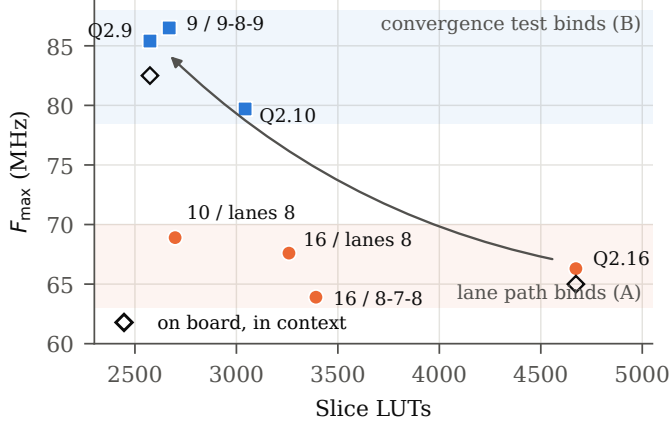


Fig. 3. $F_{\max}$ against area for the seven builds. Critical paths fall into two families with no overlap; the Q2.16→Q2.9 step crosses between them. Diamonds: the two headline builds on the board, timing met in context.

below the main width inserts a cast on the lane path, and two bits of it are enough to return the lane to the slow family: at the same 10-bit main width, 8-bit lanes reach 68.9 MHz against 79.7 for 10-bit lanes. (A 1-bit cast at $F = 9$ is not.)

Table VI and Fig. 3 show all seven builds. The four whose critical path is the lane path stay in 63.9–68.9 MHz however their lanes are set; $F_{\max}$ rises only when the lane path stops binding.

C. On the board, at speed

Out-of-context figures exclude the clock network and I/O. To test whether the headline survives a complete design, we wrapped both builds for a Basys3 board — problems from ROM, results over UART — clocked each from one MMCM at an exact frequency, and swept downward. A bitstream was written only for the highest frequency meeting setup and hold after routing, in context. Q2.16 met 65 MHz and missed 66.25; Q2.9 met 82.5 MHz and missed 85. At those clocks both builds return results bit-exact against the golden vectors for all three operators. The speed-up in context is 26.9% at the met points, bracketed +24.5% to +30.8% by the step size, overlapping the out-of-context bracket: the frequency gain belongs to the datapath, not to the measurement method. (A first bring-up used a fabric clock divider behind a LUT multiplexer; it produced correct results at 50 MHz but fails in-context timing

by 5.7 ns, because both clocks reach the clock net. We report only the MMCM builds.)

D. Lane area is linear in width

Fitting $A_{\text{lane}}(W) = bW + c$ on the three builds whose lanes equal the main width ($W = 16, 10, 9$) gives $b = 19.6$ LUT/bit per proximal instance and predicts the held-out $W = 8$ build at the same cast depth to -6.3% . A quadratic fit, exact on the same three points, predicts it to -21.6% with a negative quadratic coefficient, so we reject it: at lane granularity, casts, comparators and saturating adders dominate, and only the L2 lane contains a multiplier. A second $W = 8$ build, with deeper casts into a 16-bit main path, is predicted to -10.4% . Narrowing all lanes together pays: 8-bit lanes at $F = 16$ use 1020 LUTs against 2159 for 16-bit lanes.

E. A negative result: per-operator asymmetry

If lane area were quadratic in width, as multiplier-dominated datapaths are [12], narrowing the lanes whose operators tolerate it would pay. It is linear, so narrowing one lane by a bit saves a fraction of b , while the narrowed lane needs a cast in and out of the main width. We built matched pairs at two main widths, identical except for one narrowed lane (L1/Box/L2 at 8/7/8 vs. 8/8/8 with $F = 16$; 9/8/9 vs. 9/9/9 with $F = 9$). Asymmetry *increases* area by 4.1% and 3.7%, and the proximal lanes themselves by 13.0% and 9.3% (17 and 12 LUTs per proximal instance): the casts cost more than the bit saves. Dynamic power is identical within each pair (10 vs. 10 mW; 7 vs. 7 mW), so any difference is below the 1 mW resolution, and $F_{\max}$ moves by -5.5% and $+1.3\%$ — inconsistent in sign, the latter one search step — so we claim no frequency effect. The result is scoped to what we built: one lane narrowed by one bit, at two main widths. There it is a fabric loss at both.

F. A second problem: LASSO

LASSO, $\min \frac{1}{2} \|Ax - b\|^2 + \lambda \|x\|_1$, maps onto the unchanged datapath with the L1 lane at $\kappa = \lambda/\rho$. We registered four predictions before running it, for 2-sparse problems with Gaussian A of m rows. (i) The operator model holds on LASSO's native distribution: it gives 0.981–1.028 over five values of λ , at $d = 0.62$ – 0.77 with the threshold wherever λ puts it, where the classical model is off by 1.6 – $2.7\times$ (Fig. 1b). This matters beyond scope: on the detection workload the L1 lane sits at $d \approx 0$, so this is the first test of the $(1 - d)$ factor for L1 at the solver's own threshold, on data it produces itself. (ii) Width need grows as rows are removed and $\|M\|_2 \rightarrow 1/\rho$ (Table VII): $F^* = 8$ at $m = 32$, 9 at $m = 16$, 9–11 at $m \leq 8$; a trend over twelve settings, not a law.

(iii) *Failed*: we predicted the recovered support would match double precision on $\geq 99\%$ of instances; it matches on 90–100% per setting. After the fact, every disagreeing coefficient lies within 2 LSB of zero — ties at the format's resolution — which characterizes the failure without rescuing the prediction. (iv) Three instances run bit-exact through the unmodified RTL at $F = 16$ and $F = 9$, and on the board at $F = 9$ (50 MHz).

TABLE VII
LASSO: WIDTH F^* FOR NMSE 10^{-4} AND SUPPORT AGREEMENT WITH
DOUBLE PRECISION AT F^* (40 INSTANCES PER CELL)

m	$\ M\ _2$	$\lambda/\lambda_{\max} = 0.1$	0.2	0.3	support
32	0.74	8	8	8	100%
16	0.87	9	9	9	92.5–100%
8	0.99	10	11	9	90–100%
6	1.00	11	10	11	95–100%

VII. FOUR WAYS THESE NUMBERS WOULD HAVE BEEN WRONG

Each of the following is standard practice, each was in our own pipeline before we caught it, and each would have changed a headline.

a) *Vectorless power*: The vendor’s default estimate gave 112 and 78 mW dynamic for the two headline builds; activity-annotated measurement gives 11 and 7. The ratio survived roughly intact by luck. An earlier asymmetric/uniform pair showed an 18.8% power saving under the same estimator; measured, both are 11 mW. That result does not exist.

b) *Extrapolated $F_{\max}$* : Computing $1/(T - \text{slack})$ from builds at a loose 20 ns constraint gave 58.9 and 71.1 MHz, +20.7%. Sweeping the constraint gives 66.3 and 85.4, +28.8%; the shortcut understated both builds and the gain, because slack at one constraint does not predict what the router achieves under another.

c) *LUT cells vs. Slice LUTs*: Counting LUT cells, as scripts that query the netlist do, gives 6308 for a build that `report_utilization` reports as 4673 Slice LUTs; LUT combining packs two cells per LUT6. The design ratio barely moves (−46.6% vs. −44.9%), but the asymmetry penalty does: on cells it is +4.3% at both widths, identical to the decimal — a coincidence of the metric; on Slice LUTs, +4.1% and +3.7%. We quote Slice LUTs throughout.

d) *A bitstream is not a timing-clean design*: Our first board bring-up returned correct results at 50 MHz from a divider clock that, checked in context, fails timing by 5.7 ns setup and 2.6 ns hold. The tool flow wrote the bitstream without complaint. Our build script now refuses to write one that does not meet setup and hold, and also refuses a ROM image that does not match the golden vectors — a ROM that fails to load reads as zero, and a solver given zeros returns zeros, which compare equal.

VIII. COMPARISON WITH PRIOR ACCELERATORS

Table VIII is not a ranking, and we claim no efficiency advantage: our design solves the smallest problem on the smallest device in it, and no normalized metric is meaningful when accuracy targets go unstated. It shows two things. First, apart from Jerez et al., who bound the error, and Wu et al., who search, the fixed-point designs state their wordlength without deriving it — 18, 16, 14, 24, 32 bits — including two recent low-cost designs, a 16-bit/10-fraction GPC solver [15] and a 32-bit Artix-7 MPC solver validated against a floating-point build as having “negligible additional error” [19]. That

TABLE VIII
ADMM-CLASS FPGA ACCELERATORS. NOT A RANKING: PROBLEMS,
SIZES, DEVICES AND ACCURACY TARGETS ALL DIFFER. N/R: NOT
REPORTED.

work	device, format	LUT	MHz	power (method)
This work	Artix-7 35T, 11 b	2574	85.4	7 mW (SAIF)
Jerez [7]	Virtex-6, 18 frac	n/r	400 ^a	n/r
Peccin [15]	MAX10, 16 b (10 frac)	n/r ^b	50	n/r
ADMIN [16]	Virtex-7, 18 b	13392	263	n/r
TASER ^c [17]	Virtex-7, 14 b	4790	232	600 mW (est.)
Wu [14]	ZCU106, 20→24 b	6807	403	307 mW (XPE)
Hamadouche [18]	ZCU106, 24 b	1822	157	44 mW (est.)
AccelMPC [19]	Artix-7 100T, 32 b	n/r	100	n/r
RSQP [20]	Alveo U50, fp32	24245	300	19 W (meas.)
Zhang [21]	ZCU102, fixed/float	147238	250	10.7 W

^a Assumed, not a met constraint. ^b 30 662 Intel LEs, not convertible to Slice LUTs.

^c Forward-backward splitting, not ADMM.

decision is what an error model can inform. Second, power is usually absent or estimated; ours is activity-annotated and post-route.

IX. LIMITATIONS

a) *Ensemble, not instance*: The loop-level relation predicts the mean over an ensemble at given conditioning. Individual instances can need more: searching for bad right-hand sides, we found L1 instances under-predicted by +7.6 bits from random draws and +11.4 from adversarial search (+0.7 and +1.1 for L2). The model guides the width; bit-exact simulation over representative instances confirms it, as here. A guaranteed bound is available from [7], [8] at the conservatism our model avoids; the designer chooses.

b) *Loop gain for L1 and Box*: Only the operator-level model is claimed for these lanes; their loop-level fits did not generalize to disjoint instances, and no mechanism is offered for the difference between them.

c) *Scope*: One problem size ($N = 8$), one device family, two problem classes. Power figures carry ± 0.5 mW of rounding.

X. CONCLUSION AND ARTIFACT

Proximal operators do not pass quantization error through; on their degenerate sets they annihilate it, and the fraction annihilated is measurable. Modelling that predicts operator error to within 1% on average for soft-threshold and box projection, and within 3% on a second problem; it misses the one operator without a degenerate set by 12%. The Q2.9 design it explains saves 44.9% of LUTs and 36% of dynamic energy per solve, with unchanged detection BER, while running faster, on a board, at speed. Spending the saved bits on one operator’s lane, however, does not pay. The RTL, bit-exact model, every analysis and synthesis script, and the reports behind each number will be released with the paper; the model-side analyses and all seven builds reproduce exactly from them.

REFERENCES

- [1] S. Boyd, N. Parikh, E. Chu, B. Peleato, and J. Eckstein, “Distributed optimization and statistical learning via the alternating direction method of multipliers,” *Foundations and Trends in Machine Learning*, vol. 3, no. 1, pp. 1–122, 2011.
- [2] B. Widrow and I. Kollár, *Quantization Noise: Roundoff Error in Digital Computation, Signal Processing, Control, and Communications*. Cambridge University Press, 2008.
- [3] J. A. López, C. Carreras, and O. Nieto-Taladriz, “Improved interval-based characterization of fixed-point LTI systems with feedback loops,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 26, no. 11, pp. 1923–1933, 2007.
- [4] J.-C. Naud, D. Ménard, G. Caffarena, and O. Sentieys, “A discrete model for correlation between quantization noises,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 59, no. 11, pp. 800–804, 2012.
- [5] V.-P. Ha, T. Yuki, and O. Sentieys, “Towards generic and scalable word-length optimization,” in *Design, Automation & Test in Europe Conference (DATE)*, 2020.
- [6] V.-P. Ha and O. Sentieys, “Maximizing computing accuracy on resource-constrained architectures,” in *Design, Automation & Test in Europe Conference (DATE)*, 2023.
- [7] J. L. Jerez, P. J. Goulart, S. Richter, G. A. Constantinides, E. C. Kerrigan, and M. Morari, “Embedded online optimization for model predictive control at megahertz rates,” *IEEE Transactions on Automatic Control*, vol. 59, no. 12, pp. 3238–3251, 2014.
- [8] A. B. Kinsman and N. Nicolici, “Automated range and precision bit-width allocation for iterative computations,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 30, no. 9, pp. 1265–1278, 2011.
- [9] N. Parikh and S. Boyd, “Proximal algorithms,” *Foundations and Trends in Optimization*, vol. 1, no. 3, pp. 127–239, 2014.
- [10] A. Hamadouche, Y. Wu, A. M. Wallace, and J. F. C. Mota, “Sharper bounds for proximal gradient algorithms with errors,” arXiv:2203.02204, 2022.
- [11] —, “Probabilistic verification of approximate algorithms with unstructured errors: Application to fully inexact generalized ADMM,” arXiv:2210.02094, 2022.
- [12] G. A. Constantinides, P. Y. K. Cheung, and W. Luk, “Wordlength optimization for linear digital signal processing,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 22, no. 10, pp. 1432–1442, 2003.
- [13] C. J. Li, X. Li, B. Lou, C. T. Jin, D. Boland, and P. H. W. Leong, “Fixed-point FPGA implementation of the FFT accumulation method for real-time cyclostationary analysis,” *ACM Transactions on Reconfigurable Technology and Systems*, vol. 16, no. 3, 2023.
- [14] Y. Wu, A. M. Wallace, J. F. C. Mota, A. Abmann, and B. Stewart, “Efficient reconfigurable mixed precision ℓ_1 solver for compressive depth reconstruction,” *Journal of Signal Processing Systems*, vol. 94, pp. 1083–1099, 2022.
- [15] V. B. Peccin, D. M. Lima, R. C. C. Flesch, and J. E. Normey-Rico, “Fast constrained generalized predictive control with ADMM embedded in an FPGA,” *IEEE Latin America Transactions*, vol. 18, no. 2, pp. 422–429, 2020.
- [16] S. Shahabuddin, I. Hautala, M. Juntti, and C. Studer, “ADMM-based infinity-norm detection for massive MIMO: Algorithm and VLSI architecture,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 29, no. 4, 2021.
- [17] O. Castañeda, T. Goldstein, and C. Studer, “Data detection in large multi-antenna wireless systems via approximate semidefinite relaxation,” *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 63, no. 12, 2016.
- [18] A. Hamadouche, Y. Wu, A. M. Wallace, and J. F. C. Mota, “A low-power hardware-friendly optimisation algorithm with absolute numerical stability and convergence guarantees,” arXiv:2306.16935, 2023.
- [19] A. Grillo and B. Plancher, “AccelMPC: High-rate, low-power FPGA-accelerated model predictive control for tiny drones,” arXiv:2609.09380, 2026.
- [20] M. Wang, I. McInerney, B. Stellato, S. Boyd, and H. K.-H. So, “RSQP: Problem-specific architectural customization for accelerated convex quadratic optimization,” in *International Symposium on Computer Architecture (ISCA)*, 2023.
- [21] Y. Zhang *et al.*, “A sparsity-aware autonomous path planning accelerator with HW/SW co-design,” *ACM Transactions on Architecture and Code Optimization*, 2025.